

Worksheet: RDF, RDFS & OWL

Foundations of Artificial Intelligence – SS 2026 *University of Bremen,
Institute for Artificial Intelligence*

Instructions: Written tasks () should be answered directly in this document or in a Markdown cell in the notebook. Programming tasks () belong in `rdf_owl_exercises.ipynb`. Estimated time: **~50 minutes**.

Part 1 – RDF Triples (~10 min)

1.1 The following statements are given in natural language. Write them as RDF triples in Turtle syntax. Use the prefix `@prefix ex: <http://example.org/>` and `@prefix xsd: <http://www.w3.org/2001/XMLSchema#>`.

1. Marie Curie is a person.
2. Marie Curie was born on 07.11.1867.
3. Marie Curie discovered Radium.
4. Radium is a chemical element.
5. Marie Curie is the same person as `<http://dbpedia.org/resource/Marie_Curie>`.

1.2 In the RDF data model, which of the following are valid positions for **Literals**? Mark all that apply.

- (a) Subject
- (b) Predicate
- (c) Object

1.3 Rewrite the following two triples using Turtle **shorthand notation** (semicolon shortcut):

```
ex:Paris ex:isA ex:City .  
ex:Paris ex:locatedIn ex:France .
```

1.4 What is the purpose of **Blank Nodes** in RDF? Give a short example using the square-bracket notation to represent: *“Alice weighs 62 kg.”*

Part 2 – RDFS (~8 min)

2.1 Given the following RDFS statements, what can be inferred about `:Michaela`? Explain each inference step.

```
:Michaela rdf:type :Teacher .  
:Teacher rdfs:subClassOf :Person .  
:Person rdfs:subClassOf :LivingBeing .
```

2.2 Complete the RDFS snippet below. Fill in the blanks (___) to correctly model: “A professor teaches courses. Teaching is a sub-property of instructing.”

```
:Professor rdf:type ___ .
:teachesCourse rdf:type rdfs:Property .
:teachesCourse rdfs:domain ___ .
:teachesCourse rdfs:range ___ .
:teachesCourse ___ :instructs .
```

2.3 State **two things** that RDFS *cannot* express, and briefly explain why each requires OWL.

Part 3 – OWL Properties (~12 min)

For each scenario below, name the **OWL property characteristic** that best models it, and write a short Turtle snippet demonstrating it.

- (a) If Alice is a sibling of Bob, then Bob is a sibling of Alice.
 - (b) If Alice is taller than Bob and Bob is taller than Carol, then Alice is taller than Carol.
 - (c) A person can only have one biological mother.
 - (d) `:bornIn` is a property such that knowing someone was born in a place uniquely identifies the person (assume each person is born in exactly one place, and we use this to infer `owl:sameAs`).
 - (e) `:worksFor` and `:employerOf` express the same relationship from opposite directions.
-

Part 4 – OWL Restrictions (~10 min)

4.1 Write OWL restrictions in Turtle for the following statements:

- (i) Every `Pizza` has exactly 1 base (property `:hasBase`).
- (ii) A `VeganPizza` is a `Pizza` where all toppings (`:hasTopping`) come from the class `:VeganTopping`.
- (iii) A `MeatLover` is a `Pizza` that has at least one topping from `:MeatTopping`.

4.2 Consider this OWL snippet:

```
:capitalOf a owl:InverseFunctionalProperty .
:Berlin :capitalOf :Germany .
:Hauptstadt :capitalOf :Germany .
```

What can an OWL reasoner infer from this? Explain.

Part 5 – Spotting Mistakes (~8 min)

Each snippet below contains **one mistake**. Identify and correct it.

(a)

```
ex:hasName rdf:type owl:ObjectProperty .
ex:Bob ex:hasName "Robert" .
```

(b)

```
ex:Doctor rdfs:subClassOf ex:Person .
ex:Person rdfs:subClassOf ex:Doctor .
```

(Hint: think about what a reasoner would conclude.)

(c)

```
ex:Cat owl:disjointWith ex:Animal .
ex:Whiskers rdf:type ex:Cat .
```

(Hint: what class does every OWL individual implicitly belong to?)

Part 6 – Python: Simple RDF Graph Builder (~12 min)

Open (or create) `rdf_owl_exercises.ipynb` and complete the following tasks.

6.1 Using Python's `rdflib` library, build a small RDF graph for the following facts:

- `:Alice` is a `:Person`
- `:Alice` `:knows` `:Bob`
- `:Bob` is a `:Person`
- `:Alice` `:hasAge` 30 (as an integer literal)
- `:Bob` `:worksAt` `:UniBremen`
- `:UniBremen` is a `:University`

```
from rdflib import Graph, Namespace, Literal, RDF
from rdflib.namespace import XSD
```

```
EX = Namespace("http://example.org/")
g = Graph()
g.bind("ex", EX)
```

```
# Add your triples here ...
```

```
print(g.serialize(format="turtle"))
```

6.2 Query the graph: print all `:Person` instances using a simple `for` loop over `g.triples()`.

6.3 Serialize the graph to a file `knowledge_base.ttl` and reload it to verify it round-trips correctly.

Good luck!